

Strategies for joining the big league of cloud orchestration

Cloud orchestration has experienced various waves of development. In the first wave, infrastructures were fairly static. The second wave introduced more complexity with the use of containers, Docker and Datadog. Modern cloud infrastructure is even more complex, leveraging serverless and managed services as part of cloud applications.

It is important to note that all major cloud providers have their own version of IaC, using different languages and syntax to render code. Table 1 provides an overview.

Developers have gained enormous potential and convenience from the cloud with its elastic capabilities and API based architectures. However, GitHub and Bitbucket have also become valuable sources for attackers. These attackers leverage automation to quickly and efficiently harvest stray credentials, creating more opportunities.

In the current environment, automation is omnipresent, particularly in the form of ALOps, robotic process automation (RPA), and other machine-driven tools. Consequently, a multi-cloud environment needs higher levels of automation.

Configuration tracking and capabilities are becoming essential in the modern cloud context, especially in sectors like healthcare. Cloud administrators often need to detect idle users, disconnect them, drain hosts, and shut down unnecessary servers to minimise runup costs.

They need to monitor service accounts, cloud billing accounts and side effects of automation such as large scale cloud costs.

IaC tools for cloud orchestration

IaC tools include AWS CloudFormation, Red Hat Ansible, Chef, Puppet, SaltStack and HashiCorp Terraform. Some tools rely on domain-specific languages (DSLs), while others use standard template formats like YAML and JSON.

Cloud Provider	IaC	Language used
Google Cloud Platform	Deployment Manager	YAML
Amazon Web Services (AWS)	Cloud Formation	YAML or JSON
Microsoft Azure	Azure Resource Manager	JSON

Table 1: The different versions of IaC developed and used by the major cloud providers

Terraform: Terraform is a declarative provisioning and infrastructure orchestration tool that allows engineers to automate all aspects of their cloud-based infrastructure.

It works with all the leading cloud providers and enables the automation of resource deployment across multiple providers in parallel, regardless of the physical location of servers, DNS servers, or databases. It can handle applications written in any programming language.

SCARLETEEL, a recent cyber attacker, is an excellent example of how IaC machinery was exploited to expand its reach. In this case, Terraform was used by the cloud administration, and files were stored in an S3 bucket.

Another way to define IaC is by using a well-known programming language. For instance, Pulumi can be used to write IaC using TypeScript, JavaScript, Python, Go, and .NET.

Red Hat's Ansible: Red Hat's Ansible is an open source automation platform that enables IT automation across cloud, on-premises, and virtualised and containerised environments. It reduces human capital, and develops more repeatable and standardised processes. Its use of the YAML language makes it more accessible. The open source model of Ansible allows anyone to contribute

ideas and share them across the Ansible user community.

IBM, which acquired Red Hat for US\$ 34 billion in 2019, has integrated Ansible into its Global Technology Services (GTS) organisation, a division of IBM Services.

SaltStack: SaltStack, also known as Salt, is a configuration management and orchestration tool. It uses a central repository to provide new servers and make changes to existing ones. It also helps with software installation in various IT environments, including physical and virtual servers, as well as the cloud.

Chef: Terraform is a provisioning tool, while Chef is a configuration management tool. Terraform provisions the servers, while Chef manages the software already installed on the servers.

Puppet: Puppet allows the definition of required software and configuration for a system, maintaining a specified state after the initial setup. It uses a declarative domain specific language (DSL) that is similar to Ruby to define configuration parameters for specific environments or infrastructures.

The future of IaC depends upon technological advancements and the attitudes of cloud administrators. The objective of cloud automation is not to replace humans with machines but to make work life more meaningful. 

References

- 'What is IaC? (Tutorial)' by Beau Carnes
- 'Introduction to IaC in Multi-Cloud Environment'; onpointcorp.com/
- IaC market information by marketsandmarkets.com
- 'What is Infrastructure as Code (IaC)?' as published in ibm.com
- Certain articles related to IaC carried in computerweekly.com

 By: Vinayak R. Adkoli

The author holds a B.E. in industrial production, and has served as a lecturer in three different polytechnics for 10 years. He is also a freelance writer and cartoonist.